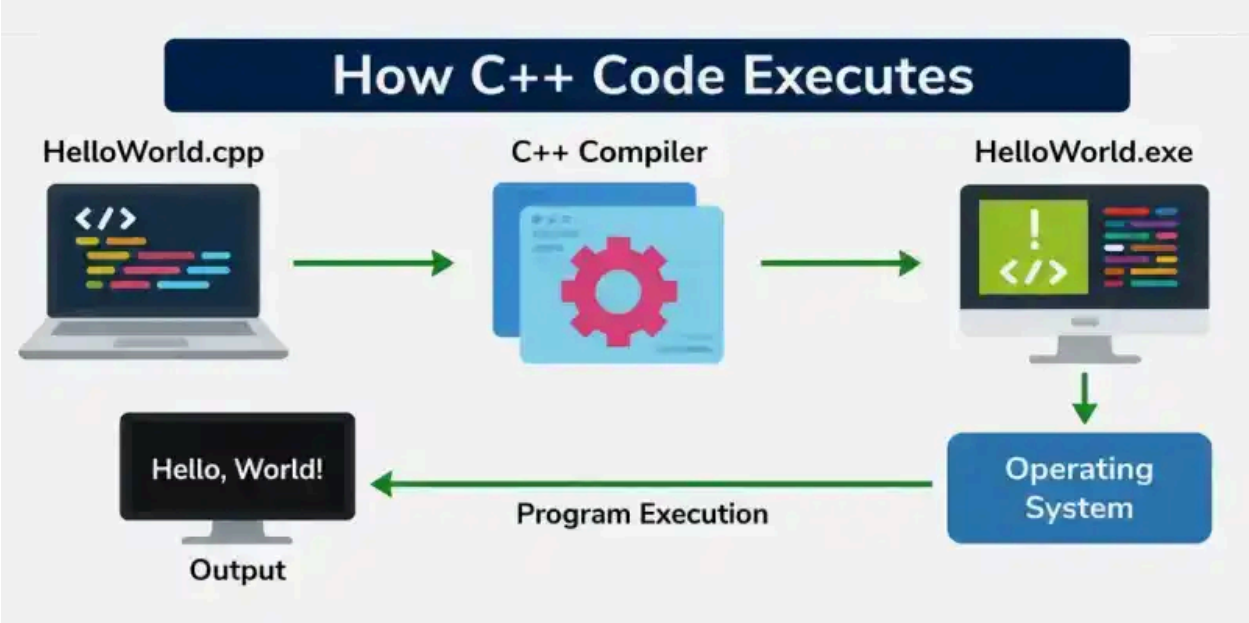
Your First C++ code:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

Code	What It Does
`#include <iostream>`	Includes the standard input/output tools needed for `cin` and `cout`.
`using namespace std;`	Lets us write `cout` instead of `std::cout` in this example.
`int main()`	Defines the main function where program execution begins.
`{ }`	Marks the beginning and end of a block.
`cout`	Prints output to the screen.
`endl`	Ends the line and flushes the output stream.
`return 0;`	Indicates successful completion of the program.



Syntax:

C++ is case-sensitive: `main`, `Main` and `MAIN` are different identifiers.
Most statements end in a semicolon.
Braces group statements into blocks.
Keywords such as `int`, `if`, `else`, `return` and `while` have predefined meanings.
Identifiers are names we choose for variables and functions.

Data Types:

Type	Stores	Example
`int`	Whole numbers	`int marks = 90;`
`long long`	Larger integer values	`long long n = 10000000000LL;`
`float`	Decimal values, lower precision	`float x = 3.5f;`

`double`	Decimal values, generally higher precision	`double pi = 3.14159;`
`char`	A single character	`char grade = 'A';`
`bool`	`true` or `false`	`bool passed = true;`
`string`	Text	`string name = "Riya";`

- Use single quotes for a character (`'A'`) and double quotes for a string (`"A"`).
- A variable name cannot begin with a digit and cannot be a C++ keyword.
- Choose a type that fits the value; integer types cannot store fractional parts.

https://www.geeksforgeeks.org/cpp/cpp-data-types/

Operators:

https://www.geeksforgeeks.org/cpp/operators-in-cpp/

Functions:

Function Declaration vs Definition

A function declaration introduces a function to the compiler by specifying its return type, name, and parameters without the body, and is used when the function is defined later or in another file.

```
// Declaration
int add(int, int);
```

Function definition contains the actual code that specifies what the function does when it is called.

```
//Definition
int add(int a, int b) {
return a + b;
}
```

Input and output

```
int a, b;
cin >> a >> b;
cout << "Sum = " << a + b << endl;
```

- cin >> a >> b` reads two whitespace-separated values.
- cout <<` can chain text and values.
- cin >> name` reads one whitespace-separated word. Use `getline(cin, name)` for a full line; handle the leftover newline when mixing it with `cin >>`.

Loops

https://www.geeksforgeeks.org/cpp/cpp-loops/

For loop — useful when the number of repetitions is known or controlled by a counter.

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}
// Output: 1 2 3 4 5
```

While loop — repeats as long as its condition is true.

```
int i = 1;
while (i <= 5) {
    cout << i << " ";
    i++;
}
```

Do-while loop — body executes at least once because the condition is checked afterward.

```
int i = 1;
do {
    cout << i << " ";
    i++;
} while (i <= 5);
```

- Loop parts: initialization → condition check → body → update → repeat.
- `i < n` excludes n; `i <= n` includes n. Choose the boundary intentionally.
- Forgetting to update the loop variable can create an infinite loop.
- Dry-run loops in a table: iteration, variable value, condition, output.

Conditional statements

```
if (marks >= 90) {
    cout << "Grade A";
} else if (marks >= 75) {
    cout << "Grade B";
} else if (marks >= 40) {
    cout << "Pass";
} else {
    cout << "Fail";
}
```

- `if` runs when its condition is true; `else` handles the remaining case.
- `else if` checks another condition only if earlier conditions failed.
- Condition order matters: check higher grade thresholds before lower ones.
- Use `&&` when both conditions must be true and `||` when at least one must be true.

Jump statements

Statement	Meaning
break	Immediately exits the nearest enclosing loop or switch.
continue	Skips the remaining body of the current loop iteration and proceeds to the next iteration.
return	Exits the current function; can pass a value back to the caller.

Programs to practice:

1. Read two numbers and print their sum.
2. Check whether a number is positive, negative, or zero.
3. Print the multiplication table of a given number.
4. Calculate the sum of integers from 1 to n.
5. Count the digits of a positive integer.